

Multi-Asset Next-Day Return Prediction

Bitcoin, Gold and Silver — a leakage-audited, walk-forward-validated comparison of statistical, tree-based and recurrent models against the random-walk baseline, with a live prediction dashboard and API

Final Year Project Report

Bachelor of Science in Computer Science

Prepared by: Alyan Shahid (team *teamlocalhost*)

Supervisor: _____

Department of Computer Science, University of Lahore

BSCS Fall 2022 – 2026

Date: 12 September 2026

Repository layout, methodology, feature dictionary, results, limitations and viva notes are also provided as Markdown under docs/; every table in this report is generated from the files in results/ by docs/build_report.py.

Executive Summary

This project is a machine-learning system that forecasts the **next trading day's return** of three assets — Bitcoin (BTC-USD), Gold (GC=F futures) and Silver (SI=F futures) — and presents the forecast, together with its measured reliability, in a web dashboard and a REST API. The forecast target is the next-day log return $y_t = \ln(P_{t+1}/P_t)$; the price shown to the user is recovered as $P_t \cdot \exp(\hat{y})$. Daily data from Yahoo Finance (2023-07-28 to 2026-07-28) is enriched with macro series (US Dollar Index, WTI crude, 10-year Treasury yield, S&P 500, VIX) and the crypto Fear & Greed index.

Methodology. A frozen chronological split (train $\leq$ 2025-09-10, validation $\leq$ 2026-02-17, test = 2026-02-18 $\rightarrow$ 2026-07-28) with no shuffling; a scaler fitted on training rows only; 23–24 stationary, backward-looking features; hyper-parameters and the served model chosen by 4-fold expanding-window walk-forward validation inside train+val; and a single evaluation of the untouched test set. Ten model families are compared on identical days: a random-walk baseline, historical mean, ARIMA, Ridge, Random Forest, LightGBM, CatBoost, GRU, LSTM and a stacked ensemble. Ten specific data-leakage checks are enforced by a 28-case test suite.

Result. The validation-selected model for all three assets is CatBoost (shallow, heavily regularised gradient-boosted trees). On the untouched test set its return-RMSE is within 0.3–1.5 % of the random-walk forecast (Diebold–Mariano $p = 0.15$ – 0.57), and its directional accuracy is 47.7–48.6 % (not significantly different from 50 %). **No model beats the random walk by a statistically significant margin**; the recurrent networks and ARIMA are significantly worse ($p \leq 0.05$). The tuned models converge to the unconditional drift, which is the expected outcome under weak-form market efficiency at a one-day horizon with ~ 700 – $1,000$ daily observations. The project therefore delivers a validated negative result, not a forecasting edge, and the application reports this honestly next to every forecast.

Improvement over the initial version. An audit of the original codebase found a price-reconstruction bug that corrupted every dollar metric, 30 % fabricated weekend rows for the metals, hyper-parameters selected on the test set, early stopping on training data, non-stationary price-level features, a served ensemble that predicted a constant, a broken inference path and misleading R^2 values (≈ 0.95 on price levels, which a random walk also achieves). All were fixed; the pipeline was re-run end-to-end and reproduces every reported number bit-for-bit.

Status. Working Streamlit dashboard (asset selection, indicator overlays, next-day forecast with uncertainty band, multi-model comparison, validation and test tables, figures) and FastAPI endpoints; reproducible pipeline (make pipeline); documentation for methodology, features, results, limitations and viva.

1. Introduction

Short-horizon financial forecasting is a standard test case for machine learning because it is genuinely hard: daily returns of liquid assets are dominated by noise, exhibit fat tails and volatility clustering, and any predictable component is small and unstable. Many student and published projects nevertheless report near-perfect accuracy — almost always because the evaluation is flawed (price-level R^2 , random splits, test-set tuning, look-ahead in features). This project was built as a working prediction system for Bitcoin, Gold and Silver and then subjected to a full methodological audit. The final version answers a precise question — *does any of ten model families beat the random walk at a one-day horizon, and by how much?* — with a pipeline in which every leakage path is closed and tested, and it serves the validation-selected model in a dashboard that always shows the forecast next to its held-out performance.

2. Project Objectives

- Build a reproducible, leakage-free pipeline from raw market data to next-day forecasts for three assets.
- Engineer stationary, financially motivated features from price, volume, macro and sentiment data.

- Compare baseline, classical, tree-based and deep models under expanding-window walk-forward validation, selecting hyper-parameters and the served model without touching the test set.
- Evaluate once on an untouched test period with metrics that are meaningful for returns (RMSE/MAE/R² in return space, directional accuracy with a significance test, Diebold–Mariano test against the random walk, strategy backtest).
- Deliver a demonstrable application (dashboard + API) that communicates the forecast, its horizon, the model used, its measured error and a clear disclaimer.
- Document the methodology, results, limitations and the audit-driven improvements for examination.

3. Project Scope

In scope: daily data; a one-trading-day horizon; Bitcoin (BTC-USD), Gold (GC=F front-month futures) and Silver (SI=F); free public data sources; ten model families; a Streamlit dashboard and a FastAPI service running locally or in Docker. Out of scope: intraday data, order-book or news-text features, multi-day horizons, portfolio construction, and any form of trading execution. The system is a research prototype and decision-support tool, not financial advice.

4. System Overview

The system is a single Python code base (Python 3.9, pandas, scikit-learn, LightGBM, CatBoost, TensorFlow/Keras 2.16, statsmodels, Streamlit 1.50, FastAPI) organised as a linear pipeline. Each stage is a script with a Make target; `make pipeline` reproduces every number in this report from the raw CSV files.



Figure 1. End-to-end workflow. The test split is read by exactly one script.

Stage	Module	What it does
Data collection	src/data/data_collection.py, external_data.py	Daily OHLCV for the three assets; DXY, WTI, ^TNX, ^GSPC, ^VIX; Fear & Greed index
Cleaning & features	src/data/preprocessing.py	De-duplication, trading-day calendar, log returns, indicators, ratio features, external alignment, frozen split, train-only scaler, build_dataset()
EDA	src/data/eda.py	ADF tests on price vs return, ACF/PACF, return distributions, volatility clustering
Models	src/models/registry.py, model_gru.py, model_lstm.py, ensemble_model.py	Uniform fit/predict interface for all models; two-phase fitting; stacked ensemble
Tuning	src/training/tune_models.py	4-fold expanding-window walk-forward grid search on train+val
Training	src/training/train_models.py	Phase A (train → validation metrics, loss curves, importance) and phase B (refit on train+val)
Evaluation	src/evaluation/backtesting.py, cross_validation.py, plots.py	Single test-set evaluation, model selection, results tables, figures
Serving	src/inference/prediction.py, app/streamlit_app.py, src/api/app.py	Next-day forecast with context; dashboard; REST endpoints
Quality	tests/ (19 functions, 28 cases), .github/workflows/ci.yml, Dockerfile	Look-ahead, alignment, split, reconstruction, metrics, inference tests; CI; container

Table 1. Implemented components (source of truth: the repository).

5. Dataset and Data Processing

Series	Source / ticker	Rows used	Period	Role
Bitcoin OHLCV	Yahoo Finance, BTC-USD	1,067 calendar days	2023-08-27 → 2026-07-28	target + features
Gold futures OHLCV	Yahoo Finance, GC=F	724 exchange days	2023-09-11 → 2026-07-28	target + features
Silver futures OHLCV	Yahoo Finance, SI=F	724 exchange days	2023-09-11 → 2026-07-28	target + features
US Dollar Index, WTI crude, 10-y yield	Yahoo Finance: DX-Y.NYB, CL=F, ^TNX	aligned to asset calendar	2023-08 → 2026-08	metals features
S&P; 500, VIX	Yahoo Finance: ^GSPC, ^VIX	aligned	2023-08 → 2026-08	all assets
Crypto Fear & Greed index	alternative.me API	aligned	2023-04 → 2026-08	Bitcoin feature

Table 2. Data sources. Row counts are after indicator warm-up (30 rows). Raw downloads cover 2023-07-28 → 2026-07-28.

Cleaning (`DataCleaner.clean_data`): duplicate timestamps removed; rows sorted; non-positive prices dropped. Bitcoin is re-indexed to every calendar day (no gaps were present). Gold and Silver **keep their exchange calendar** — the original pipeline forward-filled weekends, which created ~30 % synthetic zero-return rows. External series are converted to daily log returns on their own calendar and aligned to the asset's calendar with a forward fill (only past values are carried forward). After feature computation the 30-row warm-up is dropped and the frame is validated (no NaN/inf).

Exploratory analysis (training window only) confirms the modelling choices: price levels are non-stationary, returns are stationary, fat-tailed and almost uncorrelated (Table 3, Figure 3). This is why the target is the return and why the random walk is a demanding baseline.

Asset	Days (total / train)	ADF p (price)	ADF p (return)	Daily σ %	Excess kurtosis	ACF lag-1
Bitcoin	1067 / 746	0.74	< 1e-10	2.51	2.3	-0.082
Gold	724 / 504	0.98	< 1e-10	1.03	1.4	-0.040
Silver	724 / 504	0.94	< 1e-10	1.84	2.3	+0.004

Table 3. Stationarity and distribution statistics of daily log returns (`results/eda_summary.csv`).

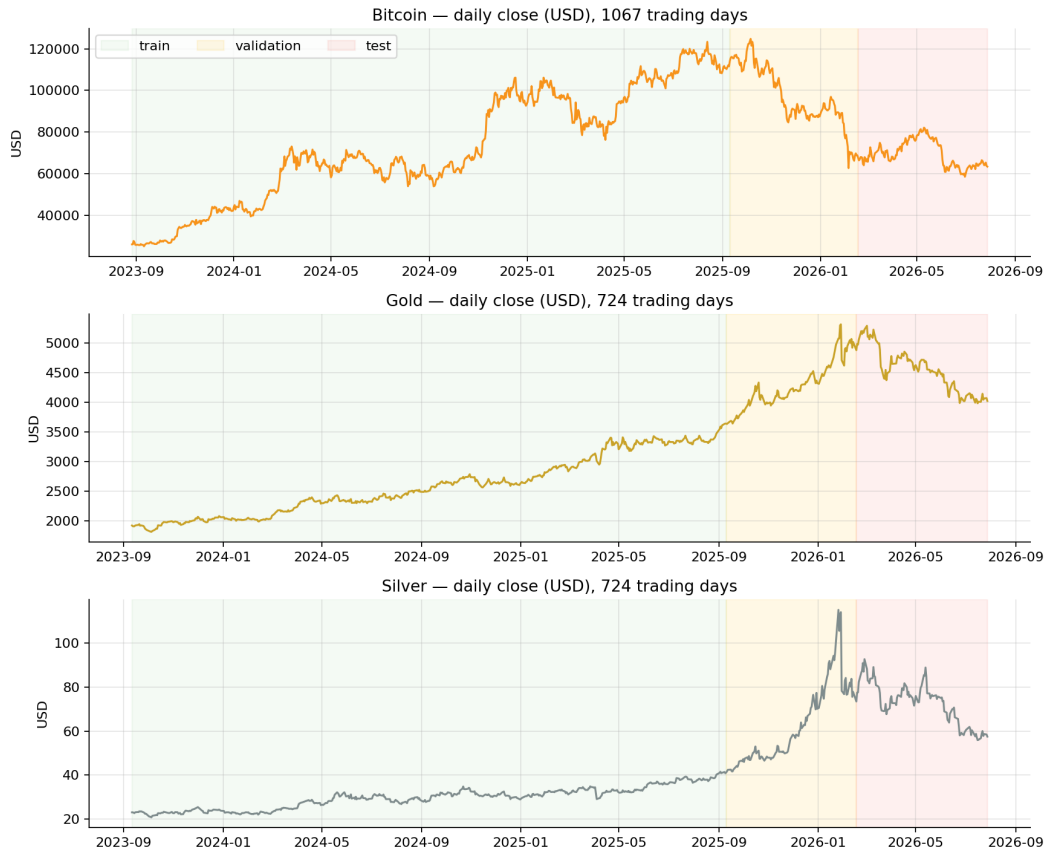


Figure 2. Daily closes with the frozen train / validation / test windows.

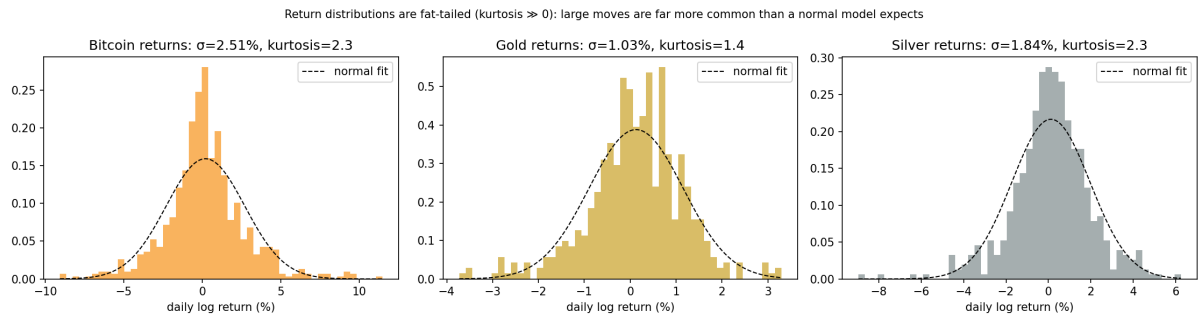


Figure 3. Return distributions are fat-tailed relative to a normal fit (training window).

6. Feature Engineering

All features at day t use data up to the close of day t . Raw price levels (open, high, low, close, volume, EMA, Bollinger bands, ATR, MACD) are retained only for charts; the models receive scale-free ratios so that the feature distribution does not drift when the price trends outside the training range (Gold's test prices, \$3,986–5,294, lie entirely above its training range of \$1,817–3,644). A unit test perturbs the last observation and asserts that no earlier feature changes (no look-ahead).

Group	Features (formula)	Assets
Return path	$\log_return = \ln(P_t/P_{t-1})$; $return_1d/2d/5d/10d$ = lagged returns; $volatility_10d/30d$ = rolling std of returns	all
Momentum / trend	$RSI(14)$; $ADX(14)$; $ROC(12)$; $macd_norm = (EMA12-EMA26)/P$; $macd_hist_norm = (MACD-Signal9)/P$; $ema14_ratio$, $ema30_ratio = P/EMA-1$	all
Volatility state	$bb_pctb = (P-BB_low)/(BB_up-BB_low)$; $bb_width = (BB_up-BB_low)/BB_mid$; $atr_norm = ATR14/P$; $hl_range = (H-L)/P$	all

Group	Features (formula)	Assets
Macro	sp500_return, vix_return (all); dxy_return, oil_return, tnx_return (metals); gold_return (silver)	per asset
Sentiment / calendar / activity	fear_greed (0–100); dow_sin, dow_cos; log_volume_change (clipped ± 3)	Bitcoin

Table 4. Model input features: Bitcoin 24, Gold 23, Silver 24 (docs/FEATURES.md gives every formula and rationale).

Deliberately excluded: futures volume for Gold/Silver (front-month contract volume from Yahoo jumps at contract rolls), day-of-week for the metals (no justification once synthetic weekends are removed), and redundant level indicators (VWAP, SMA, BB_Mid).

Target. $y_t = \ln(P_{t+1}/P_t)$, the next row's log return, created only inside `create_sequences`; the 30-day input window never contains that row (unit-tested). Recurrent models receive the $30 \times F$ window; tabular models receive the last row of the same window, so every model is scored on identical days. The target is MinMax-scaled with the features (scaler fitted on train only) and inverted exactly for reporting.

7. Machine Learning Models

Family	Model	Input	Notes
Baseline	Naive (random walk)	—	$\hat{y} = 0$, i.e. tomorrow's price = today's. The reference every model must beat.
Baseline	Historical mean	—	$\hat{y} = \text{mean training return (drift)}$.
Baseline	ARIMA(p,0,q)	return series	Order by AIC on train (BTC (1,0,0), Gold (2,0,2), Silver (3,0,2)); one-step walk-forward.
Linear	Ridge	last-day features	L2-regularised regression; α tuned.
Trees	Random Forest	last-day features	Depth-limited bagged trees.
Trees	LightGBM	last-day features	Gradient boosting, early-stopped on validation.
Trees	CatBoost	last-day features	Ordered boosting, early-stopped on validation. Served model.
Deep	GRU	$30 \times F$ window	One GRU layer (16/32 units) + dropout + dense; early stopping.
Deep	LSTM	$30 \times F$ window	Same shape as the GRU for a fair comparison.
Ensemble	Stacked	OOF predictions	Non-negative Ridge over Ridge/LightGBM/CatBoost/GRU (experiment).

Table 5. Models compared (src/models/registry.py).

Two-phase training. Every model with a stopping rule (number of trees / boosting iterations / epochs) is first fitted on the training window while monitoring validation loss to find the stopping point (phase A — this also yields honest validation metrics and loss curves), then refitted on train+validation with that stopping point fixed (phase B — the deployed model). The validation data is therefore never inside its own early-stopping monitor.

Asset	Served model & tuned parameters	Stopping point	Runner-up configurations (tuned)
Bitcoin	CatBoost: depth 3, learning-rate 0.01, L2 3.0, ≤ 500 iterations	10 iterations	LightGBM: 4 leaves, lr 0.01, λ 5.0; Ridge α 100.0; RF depth 3; GRU 32u/drop 0.2; LSTM 32u
Gold	CatBoost: depth 5, learning-rate 0.01, L2 3.0, ≤ 500 iterations	67 iterations	LightGBM: 4 leaves, lr 0.03, λ 0.0; Ridge α 100.0; RF depth 3; GRU 32u/drop 0.2; LSTM 16u
Silver	CatBoost: depth 3, learning-rate 0.01, L2 10.0, ≤ 500 iterations	10 iterations	LightGBM: 4 leaves, lr 0.01, λ 0.0; Ridge α 100.0; RF depth 3; GRU 32u/drop 0.2; LSTM 16u

Table 6. Hyper-parameters chosen by walk-forward validation (results/tuning/best_params.json). The search consistently selected the most regularised settings — shallow trees, few iterations, large α — i.e. the data supports very little model capacity.

Why CatBoost is served. Selection is automatic: the model with the lowest mean walk-forward RMSE of the return on the train+val folds. CatBoost won for all three assets, but Ridge, LightGBM and the historical mean are within 0.5 % of it and the random walk within 1 % — the ranking among these is not statistically meaningful. CatBoost was retained because it is the validation winner, trains in seconds, and exposes feature importance; the recurrent models were the worst family on every asset in validation.

8. Training and Evaluation Methodology

Split	Rule	Bitcoin	Gold / Silver	Used for
Train	$\leq 2025-09-10$	746 days (716 windows)	504 days (474 windows)	fitting; scaler
Validation	$2025-09-10 < t \leq 2026-02-17$	160 days	109 days	early stopping; walk-forward folds with train
Test	$> 2026-02-17$ (to 2026-07-28)	161 days	111 days	evaluated once by backtesting.py

Table 7. Frozen chronological split (config.py). No shuffling; identical calendar boundaries for all assets.

Walk-forward validation. The train+validation period is cut into 4 consecutive validation blocks; for fold k the model is trained on everything before the block and scored on the block (expanding window). Inside each fold the last 15 % of the fold's training window is the early-stopping monitor. Hyper-parameter grids (over regularisation strength rather than capacity) and model selection use only these folds. k -fold cross-validation was rejected because it would train on 2026 to predict 2024 and leak regime information.

Metric	Space	Why it is reported
MAE, RMSE, MAPE	USD price	What a user experiences; MAPE is scale-free across a \$60,000 and a \$60 asset.
RMSE, MAE, R^2	log return	The quantity actually predicted; $R^2 < 0$ means worse than the mean. R^2 on price levels is NOT reported because a random walk scores ≈ 0.95 there.
Directional accuracy	sign	% of non-flat days with $\text{sign}(\hat{y}) = \text{sign}(y)$, with a one-sided binomial p-value against 50 %. A zero forecast has no direction ("—").
Diebold–Mariano test	log return	Formal test of equal squared-error accuracy against the random walk (HAC variance, small-sample correction).
Strategy backtest	—	Long/flat rule (long if $\hat{y} > 0$) with 10 bps cost vs buy-and-hold — economic relevance.
Train vs validation RMSE	log return	Over-fitting gap, compared with the random walk's own gap.

Table 8. Evaluation metrics (src/utls/metrics.py).

Leakage check	Mechanism	Verified by
Random shuffling	chronological date slicing	test_split_is_chronological_and_disjoint
Scaler fitted on val/test	MinMaxScaler.fit(train) only	test_scaler_fitted_on_train_only
Target inside input window	$y = \text{row } i+30$; window = rows $i \dots i+29$	test_create_sequences_target_is_next_row_and_never_in_window
Look-ahead in rolling / EWM / shift features	all operators backward-looking	test_no_lookahead_in_any_feature
External data alignment	reindex(..., method="ffill") only	code review; alignment on asset calendar
Early stopping on training data	two-phase fit	registry.py; CatBoost bug removed
Hyper-parameters chosen on test	tuner reads train+val arrays only	grep: no test reference in training/tuning/stacking
Synthetic rows	exchange calendar for futures	test_commodities_keep_trading_calendar
Wrong price reconstruction	explicit previous close	test_true_target_reconstructs_actual_close_exactly (error $1e-11$)

Leakage check	Mechanism	Verified by
Test set touched more than once	single script	backtesting.py is the only reader of the test split

Table 9. Data-leakage checklist enforced by the test suite (tests/, 28 cases, all passing).

9. Experimental Results

Test period 2026-02-18 → 2026-07-28 for all assets (Bitcoin 161 days; Gold and Silver 111 exchange days). "CV" columns are the walk-forward validation scores used for selection; the remaining columns are the single test-set evaluation. The served model is marked "(served)". All values are from results/cv_results.csv and results/final_test_results.csv.

9.1 Bitcoin

Model	CV RMSE (ret) mean ± std	Test RMSE (ret)	Test R ² (ret)	MAE (\$)	RMSE (\$)	MAPE %	Dir. acc. % (p-value)	DM p vs naive
Naive (random walk)	0.02504 ± 0.00246	0.02064	-0.000	1,093.48	1,421.12	1.58	—	—
Historical mean	0.02522 ± 0.00259	0.02069	-0.005	1,097.44	1,425.13	1.59	47.8 (0.74)	0.57
ARIMA	0.02519 ± 0.00269	0.02084	-0.020	1,102.33	1,434.93	1.60	51.6 (0.38)	0.30
Ridge	0.02525 ± 0.00265	0.02067	-0.003	1,098.71	1,423.27	1.59	44.7 (0.92)	0.74
Random Forest	0.02555 ± 0.00266	0.02056	+0.007	1,094.69	1,414.95	1.59	52.8 (0.26)	0.60
LightGBM	0.02521 ± 0.00262	0.02067	-0.003	1,096.74	1,423.70	1.59	47.8 (0.74)	0.75
CatBoost (served)	0.02521 ± 0.00260	0.02068	-0.005	1,096.70	1,424.81	1.59	47.8 (0.74)	0.57
GRU	0.02664 ± 0.00310	0.02243	-0.182	1,210.18	1,534.17	1.75	52.2 (0.32)	0.01
LSTM	0.02627 ± 0.00363	0.02216	-0.153	1,177.22	1,515.70	1.70	52.2 (0.32)	0.03
Stacked ensemble	—	0.02064	-0.001	1,093.86	1,421.38	1.58	47.8 (0.74)	0.78

Table 10. Bitcoin: walk-forward validation and untouched test-set results, all models.

9.2 Gold

Model	CV RMSE (ret) mean ± std	Test RMSE (ret)	Test R ² (ret)	MAE (\$)	RMSE (\$)	MAPE %	Dir. acc. % (p-value)	DM p vs naive
Naive (random walk)	0.01326 ± 0.00408	0.01726	-0.010	60.97	78.84	1.34	—	—
Historical mean	0.01318 ± 0.00404	0.01749	-0.037	61.69	79.89	1.36	47.7 (0.72)	0.13
ARIMA	0.01337 ± 0.00433	0.01876	-0.194	65.74	85.90	1.45	51.4 (0.42)	0.03
Ridge	0.01316 ± 0.00402	0.01763	-0.054	62.13	80.56	1.37	47.7 (0.72)	0.16
Random Forest	0.01316 ± 0.00411	0.01778	-0.072	62.17	81.16	1.37	47.7 (0.72)	0.14
LightGBM	0.01318 ± 0.00406	0.01763	-0.054	62.00	80.51	1.37	49.5 (0.58)	0.05
CatBoost (served)	0.01312 ± 0.00406	0.01752	-0.041	61.74	80.03	1.36	47.7 (0.72)	0.15
GRU	0.01379 ± 0.00372	0.01778	-0.072	63.45	81.48	1.40	45.9 (0.83)	0.07
LSTM	0.01396 ± 0.00363	0.01847	-0.157	66.87	83.47	1.48	51.4 (0.42)	0.08
Stacked ensemble	—	0.01755	-0.045	61.99	80.22	1.37	47.7 (0.72)	0.09

Table 11. Gold: walk-forward validation and untouched test-set results, all models.

9.3 Silver

Model	CV RMSE (ret) mean $\pm$ std	Test RMSE (ret)	Test R ² (ret)	MAE (\$)	RMSE (\$)	MAPE %	Dir. acc. % (p-value)	DM p vs naive
Naive (random walk)	0.02805 $\pm$ 0.01497	0.03397	-0.004	1.95	2.57	2.65	—	—
Historical mean	0.02801 $\pm$ 0.01494	0.03415	-0.015	1.97	2.58	2.67	48.6 (0.65)	0.33
ARIMA	0.02897 $\pm$ 0.01613	0.03641	-0.154	2.14	2.76	2.90	45.9 (0.83)	0.01
Ridge	0.02797 $\pm$ 0.01494	0.03431	-0.024	1.98	2.59	2.69	42.3 (0.96)	0.17
Random Forest	0.02813 $\pm$ 0.01493	0.03438	-0.028	2.00	2.59	2.71	44.1 (0.91)	0.52
LightGBM	0.02798 $\pm$ 0.01493	0.03480	-0.054	2.04	2.63	2.77	45.9 (0.83)	0.17
CatBoost (served)	0.02797 $\pm$ 0.01497	0.03418	-0.017	1.97	2.59	2.67	48.6 (0.65)	0.27
GRU	0.03121 $\pm$ 0.01718	0.03661	-0.166	2.13	2.75	2.89	45.0 (0.87)	0.05
LSTM	0.02965 $\pm$ 0.01535	0.03601	-0.129	2.09	2.72	2.83	49.5 (0.58)	0.08
Stacked ensemble	—	0.03421	-0.018	1.97	2.59	2.68	48.6 (0.65)	0.28

Table 12. Silver: walk-forward validation and untouched test-set results, all models.

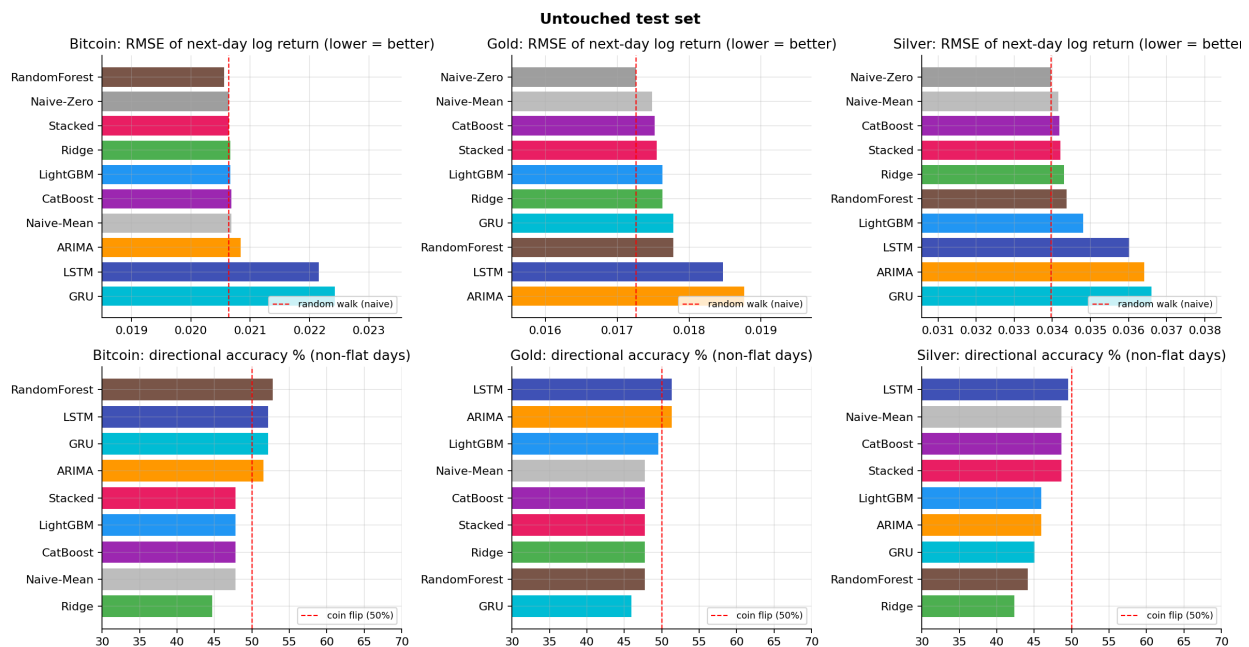


Figure 4. Test-set comparison: RMSE of the next-day return (red line = random walk) and directional accuracy (red line = 50 %).

9.4 Interpretation

- **Baselines.** The random walk is the best or joint-best forecast on every asset in both validation and test; the historical mean is within 0.2 % of it. ARIMA is significantly worse than the random walk on Gold and Silver (DM p = 0.03, 0.01): the small in-sample autocorrelations it fits do not persist.
- **Served models.** CatBoost's test return-RMSE is 0.3 % (Bitcoin), 1.5 % (Gold) and 0.6 % (Silver) above the random walk — inside the noise (DM p = 0.57 / 0.15 / 0.27). Its predictions have a standard deviation of 0.0001–0.0003 against a true return standard deviation of 0.017–0.034: the validation-selected model is effectively the unconditional drift.
- **Direction.** Directional accuracy is 42–53 % for every model on every asset and none is significant (smallest binomial p = 0.26: Random Forest on Bitcoin, 52.8 % of 161 days). Random Forest on Bitcoin also has the best test R² (+0.007) and the only positive strategy return (+3.1 % vs -6.1 % buy-and-hold), but it ranked 7th of 9 on validation; reporting it as the winner would be selection on the test set.

- **Deep models.** GRU and LSTM are the worst family in validation on every asset (R^2 -0.10 to -0.22) and significantly worse than the random walk on the Bitcoin test set (DM $p = 0.01, 0.03$), despite being reduced to a single 16–32-unit layer with dropout and early stopping. With 500–700 training windows they fit noise.
- **Stacked ensemble.** With non-negative weights on out-of-fold predictions the meta-model assigned weight 0.0 to all four base models for Bitcoin (intercept only) and only 0.04–0.16 for Gold and Silver — an independent confirmation that the base models carry no combinable out-of-sample signal.
- **Over-fitting.** Validation RMSE is $1.0\times$ (Bitcoin), $2.0\times$ (Gold) and $3.0\times$ (Silver) the training RMSE, but the random walk shows exactly the same ratios (Figure 6): the gap is regime shift — the validation and test windows are far more volatile than the training window — not memorisation.

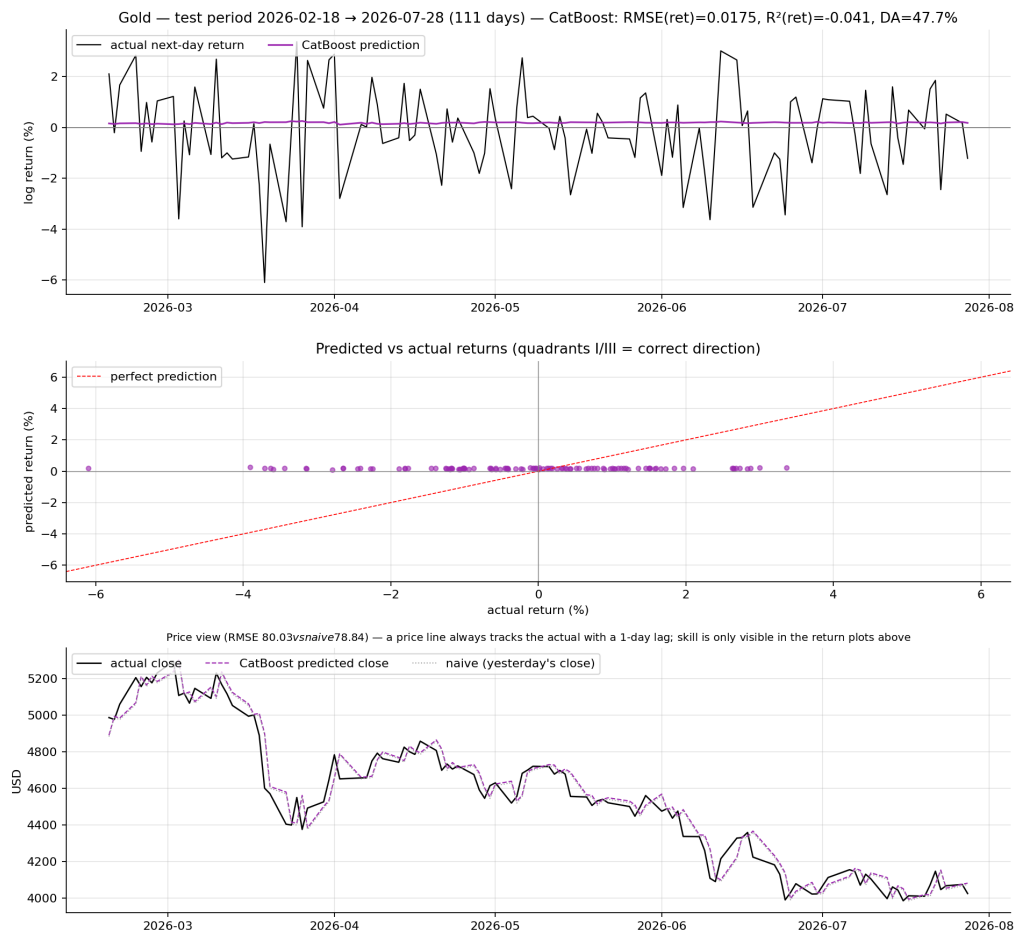


Figure 5. Gold test period, served model: predicted vs actual next-day returns (top, scatter middle) and the price view (bottom). A price line always tracks the actual with a one-day lag; skill is only visible in return space.

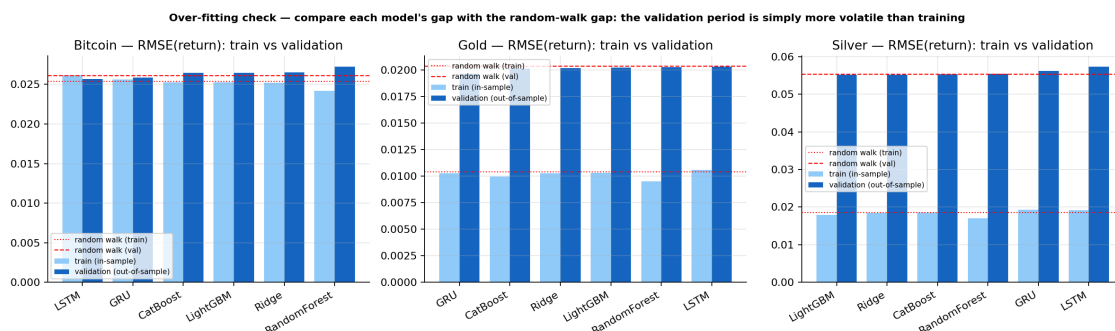


Figure 6. Train vs validation RMSE per model with the random walk's train/validation levels (dotted/dashed).

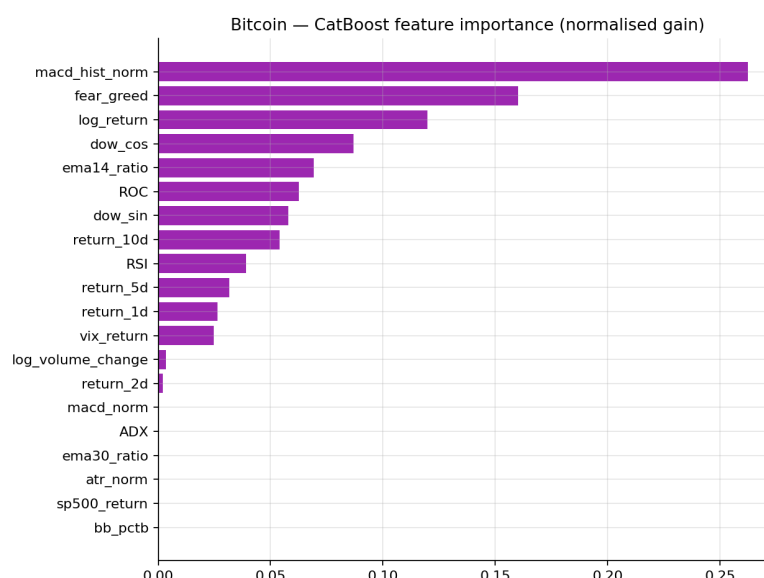


Figure 7. Bitcoin, served CatBoost: normalised gain importance. Volatility-regime and trend-distance features dominate, together with the Fear & Greed index.

10. Final System and Application

The Streamlit dashboard (make `serve`, port 8501) and the FastAPI service (make `api`, port 8000) sit on the same inference module. Inference scales the latest unscaled features with the train-fitted scaler, runs the requested model on the last 30 rows, unscales the predicted return and converts it to a price. A "Sync Live Market Data" button refreshes features into a separate live file so the frozen evaluation data is never overwritten (Yahoo Finance rate-limited during this work, so the stored data is shown).

Implemented (verified running)	Not implemented (future possibilities)
• Select Bitcoin / Gold / Silver; price chart with train/val/test shading; Bollinger, RSI, MACD, volume overlays • Run a next-day prediction with the served model or any trained model: current close, predicted close, % change, direction, horizon (T+1), model name, ± 1 RMSE uncertainty band, held-out metrics of that model, disclaimer • Forecast-horizon chart (last 60 days + next day); table of all trained models' predictions on the same input • Validation view: actual vs predicted returns and prices over the test period; return scatter • Performance tab: walk-forward and test tables for all models, honest-reading summary, report figures • Methodology tab: task, split, features, models, served-model parameters, model inventory • API: GET <code>/health</code>, GET <code>/models</code>, GET <code>/predict/{asset}[?model=...]</code>; Docker Compose for both services • Deep links: <code>?asset=Gold&run=1</code>	• Multi-day horizons or volatility forecasts • Calibrated prediction intervals (the band is an ex-post error band) • User accounts, saved forecasts, database • Automatic scheduled retraining in production • News / NLP sentiment for the metals • Intraday data • Trading execution or portfolio management

Table 13. Application functionality.

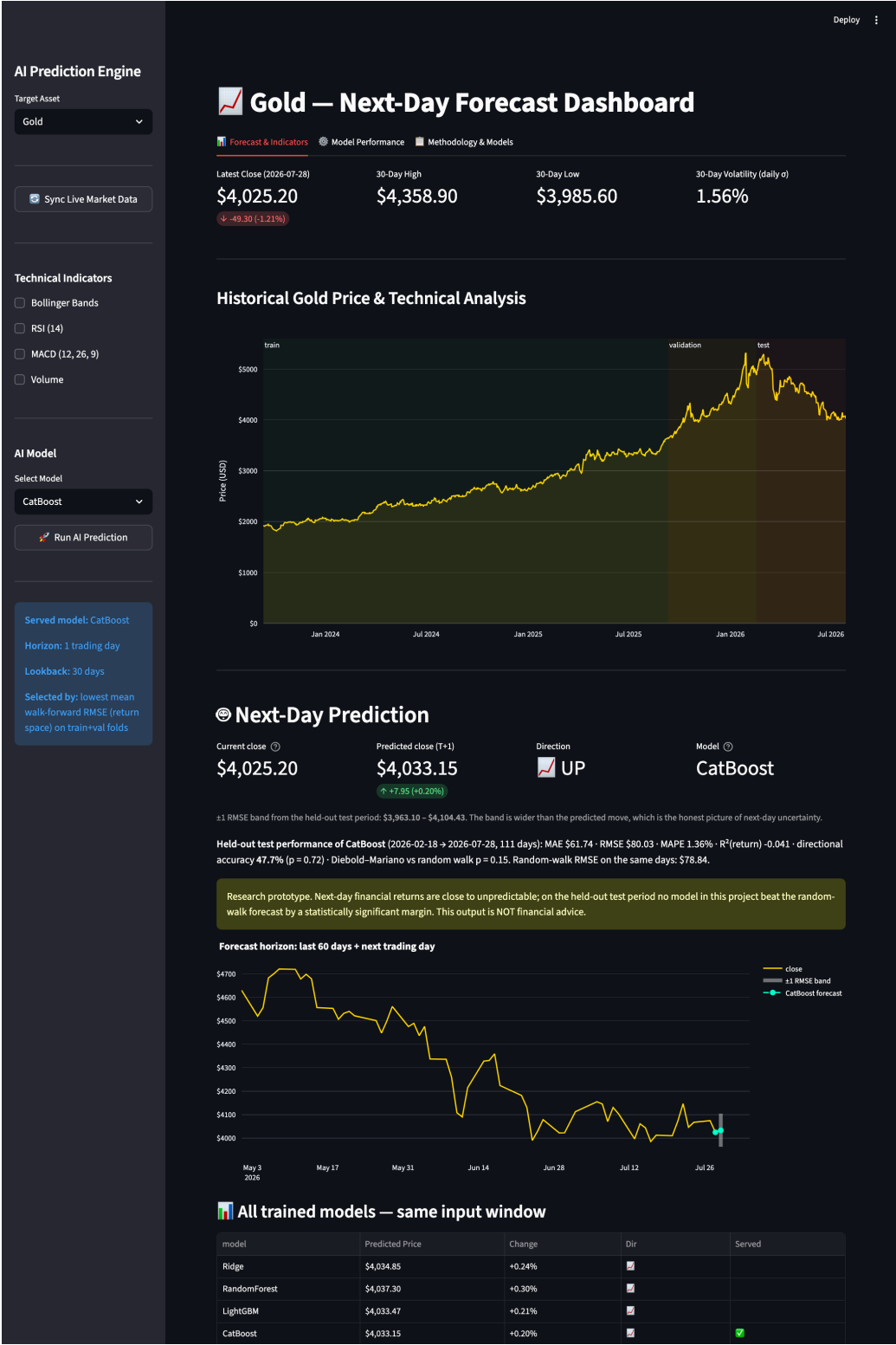


Figure 8. Dashboard, Gold: indicator chart, next-day prediction with uncertainty band, held-out metrics, disclaimer, forecast-horizon chart and multi-model table (screenshot of the running application).

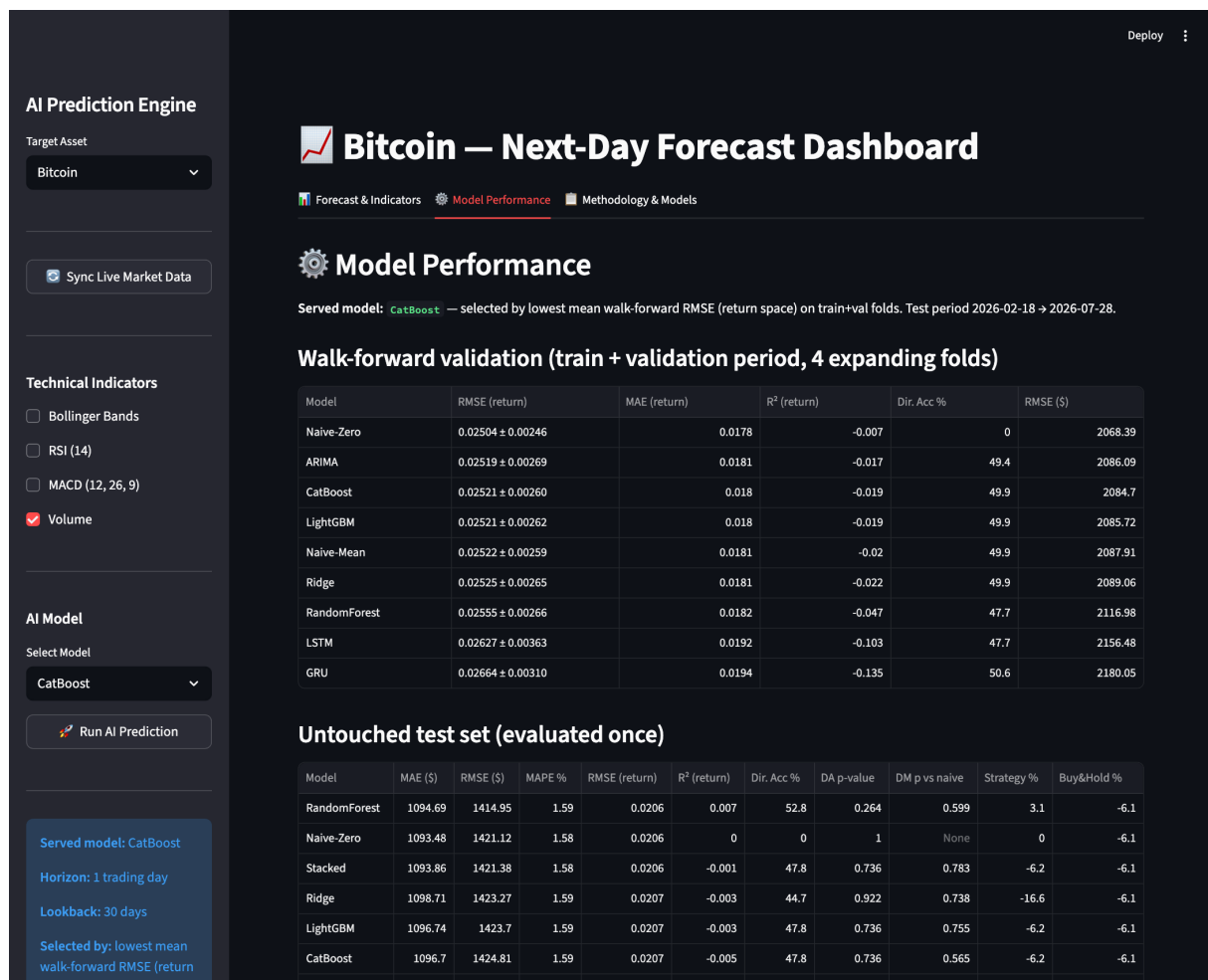


Figure 9. Dashboard, Bitcoin: Model Performance tab with the walk-forward and test tables generated by the evaluation script.

11. Improvements Made During the Audit

The initial project already generated predictions, used a log-return target, a chronological split and a train-only scaler, and had a modular code base with a dashboard. The audit nevertheless found problems that invalidated its reported results. All were fixed; the original artefacts are archived under `results/archive_pre_fix/` for the discussion.

Area	Original problem	Final implementation
Price reconstruction	Predicted price rebuilt from the previous day's open (column 0) instead of the close — every USD metric and every dashboard forecast wrong ($\approx \$1,416$ RMSE for Bitcoin from the bug alone)	Explicit previous close; regression test asserts exact reconstruction of the true target
Data pipeline	Gold/Silver forward-filled to calendar days: 30.7 % fabricated zero-return rows, 47 of 159 test days fake	Exchange calendar kept for futures; macro data aligned onto it
Data leakage	Hyper-parameters chosen on the test set (April experiments); CatBoost early-stopped on data inside its training set	Walk-forward tuner on train+val only; two-phase fitting; checklist enforced by tests
Features	Raw price levels (open/high/low/close/EMA/BB/VWAP) as inputs — test values 1.7–2.4× outside the training range	Stationary ratio features; redundant indicators removed; gold return added for silver
Model	Served "stacked ensemble" had coefficients ≈ 0 (Bitcoin exactly $[0,0,0]$) — a constant predictor, never evaluated on test; 2×100 -unit GRU/LSTM over-fitted	Uniform registry of 10 models; validation-based selection; small tunable GRU/LSTM; stack kept as a reported experiment

Area	Original problem	Final implementation
Evaluation	$R^2 \approx 0.94$ – 0.97 on price levels presented as skill; naive baseline beat every model but was not acknowledged; directional accuracy mis-computed (naive 0 %)	Return-space metrics; correct directional accuracy with p-values; Diebold–Mariano test; strategy backtest; over-fitting gap; honest reporting
Serving	CLI/API crashed (LightGBM given 900 features; float(dict)); dashboard listed stale LSTM and a non-existent Ensemble path	Rewritten inference, API and dashboard around the served model with context and disclaimer; verified in the browser
Code & docs	Five stale result files from an earlier pipeline; Makefile/CI referenced missing scripts; 4 of 31 tests failed; README claimed models/indicators that did not exist	Single results/ tree; working Makefile/CI/retrain; 28 passing tests; accurate README and docs; bit-identical reproducibility

Table 14. Before vs final implementation (docs/FIX_PLAN.md, docs/AUDIT_SUMMARY.md).

12. Limitations

- **Market efficiency dominates.** Lag-1 autocorrelation of returns is -0.08 / -0.04 / $+0.00$ (BTC / Gold / Silver); at a one-day horizon the predictable component is small and unstable, and the results are consistent with that.
- **Small sample.** Three years of daily data (Yahoo Finance rate-limited a longer download; config.DATA_START_DATE = 2018-01-01 is supported by the pipeline). With 111–161 test days, differences of a few percent in RMSE and directional accuracies of 52–56 % are not statistically detectable.
- **Regime shift in the test window.** Gold and Silver traded far above their training range and Silver's 30-day volatility exceeded every training value on 89 % of test days; one five-month window is a single draw from a non-stationary process.
- **Data quality.** Front-month futures (roll effects; unusable volume), a composite BTC index, forward-filled macro series on holidays, and a proprietary sentiment index.
- **Modelling.** One-day point forecasts only; the dashboard's band is an ex-post RMSE band, not a calibrated interval; small tuning grids; selection by RMSE (directional accuracy and Sharpe are reported but not optimised).
- **Backtest.** The long/flat strategy uses a flat 10 bps cost with no slippage, shorting, sizing or financing.

13. Future Work

- Download the full history from 2018 ($\approx 3\times$ more data) and re-run the unchanged pipeline.
- Forecast volatility (GARCH-type or quantile targets), where the volatility-regime features clearly carry information, and score calibration.
- Multi-day horizons and probabilistic forecasts; regime-aware or switching models.
- News/NLP sentiment for the metals; intraday data; a realistic transaction-cost model.

14. Conclusion

The project set out to build a working next-day prediction system for Bitcoin, Gold and Silver and to make it scientifically defensible. The final system has a leakage-audited, reproducible pipeline; ten model families compared under expanding-window walk-forward validation on identical days; a single, statistically tested evaluation on an untouched period; and a dashboard and API that serve the validation-selected model together with its measured error. The empirical answer is a validated negative result: with daily price, volume, macro and sentiment features and ~700–1,000 observations, no statistical, tree-based or recurrent model beats the random walk at a one-day horizon by a significant margin, and the models that deviate most from the drift are significantly worse. The value of the work lies in demonstrating that result rigorously — and in showing why the original " $R^2 \approx 0.95$ " was an illusion — rather than in a forecasting edge.

15. FYP Readiness Assessment

Suitable for submission and live demonstration. The implementation demonstrates the complete ML research workflow (problem → data → analysis → features → baselines → models → validation → experiments → comparison → final model → system → evaluation → limitations); every methodological claim is backed by a test, a script or a results file; the application runs for all three assets; and the pipeline reproduces every reported number. The main remaining weaknesses are the data volume and the absence of a positive forecasting result — the former is addressable by re-downloading data, the latter is the honest finding of the study and is presented as such. It should be presented as a methodological benchmark and decision-support prototype, not as a market-beating predictor.

Appendix — Reproduction

```
python -m venv venv && source venv/bin/activate && pip install -r requirements.txt
make preprocess eda tune train stack evaluate figures test # full pipeline from data/raw
make serve # dashboard on http://localhost:8501
make api # FastAPI on http://localhost:8000/docs
python docs/build_report.py # regenerate this report from results/
```

Repository documents: docs/METHODOLOGY.md, docs/FEATURES.md, docs/RESULTS.md, docs/LIMITATIONS.md, docs/VIVA_QA.md, docs/REPORT_STRUCTURE.md, docs/FIX_PLAN.md, docs/AUDIT_SUMMARY.md, results/FINAL_RESULTS.md.